

Computability 1

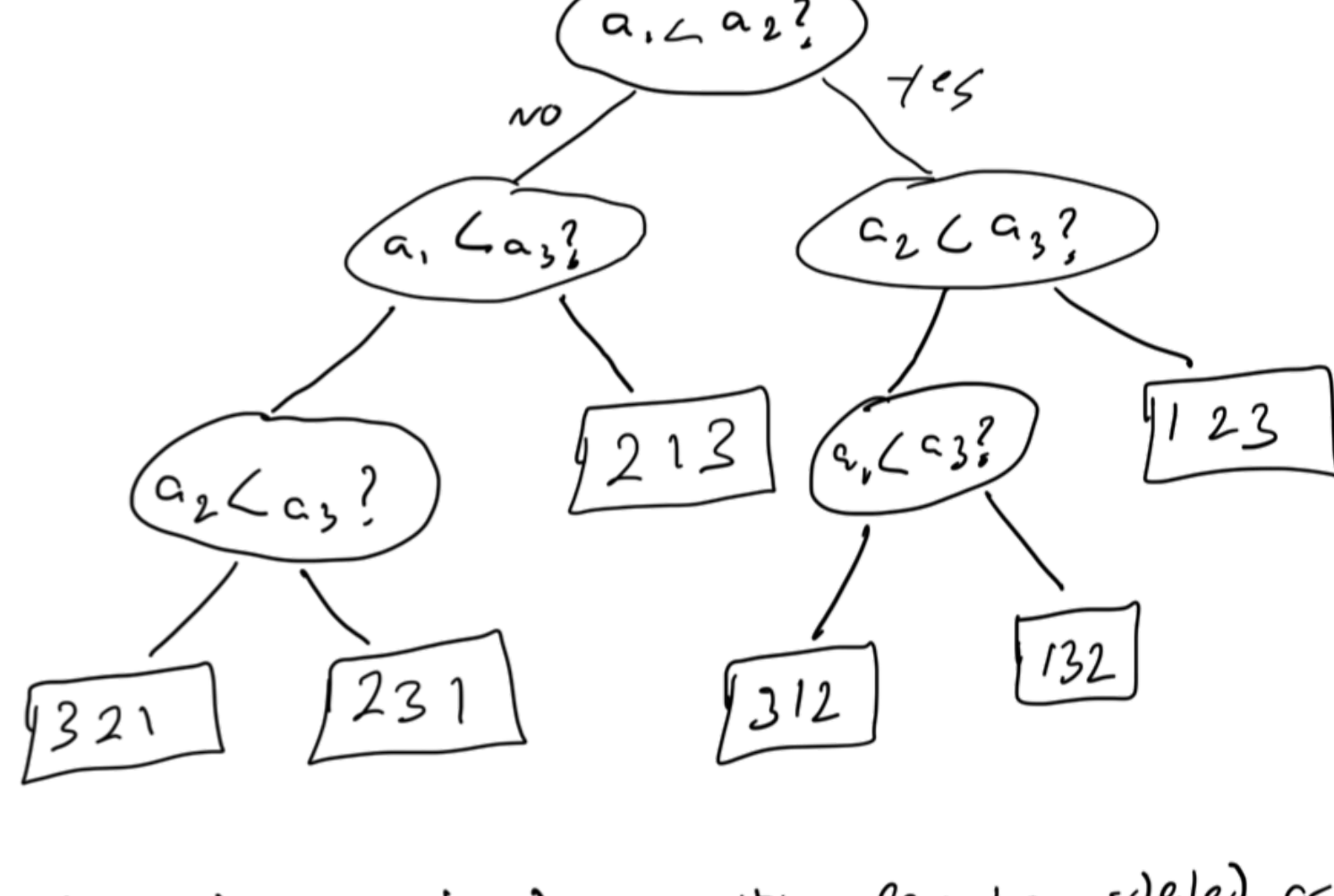
Asymptotic Complexity of a Problem rather than on algorithm

How do we know that no faster algorithm can exist?

Sorting example

Consider a sorting problem as a decision tree

- Internal nodes: a comparison of 2 elements i, j
- Branches: result of comparison
 - Left: $i \leq j$
 - Right: $i > j$
- Leaf nodes: possible final result
- Path: execution of an algorithm on a unique problem

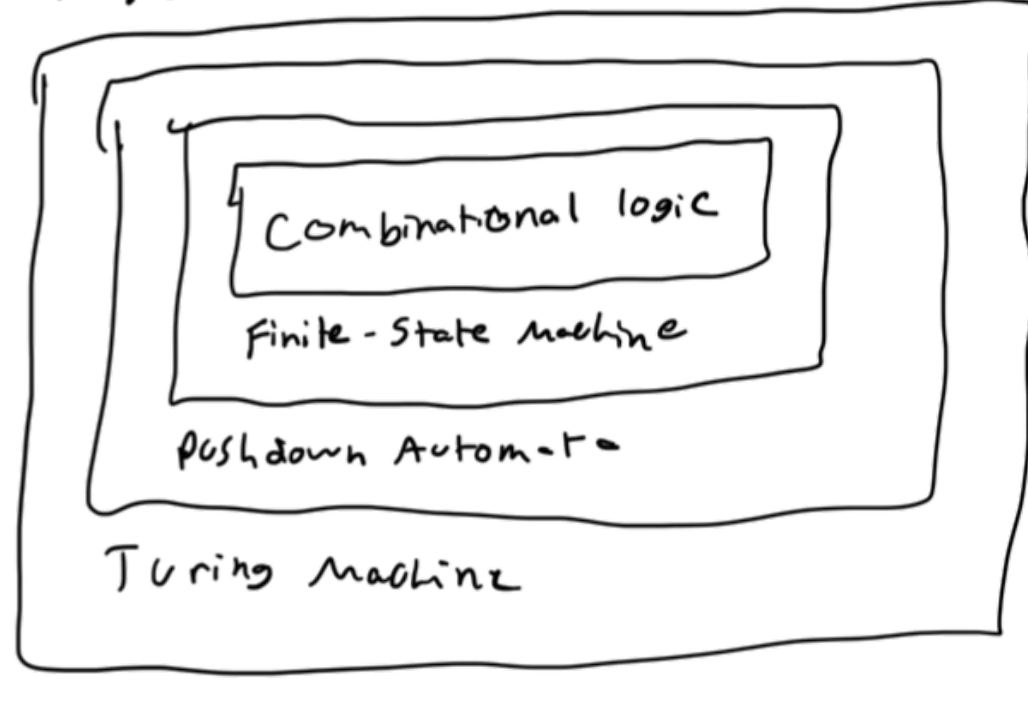


Any decision-based algorithm can be modeled as a decision tree

- worst case: longest path
- we never repeat comparisons on the same path: $(\frac{n}{2})$
- Best case:
- Every possible output must be represented as a leaf
 - $\geq n!$
- Binary tree!
- height $\approx \log_2(n!) = \Omega(n \log n)$

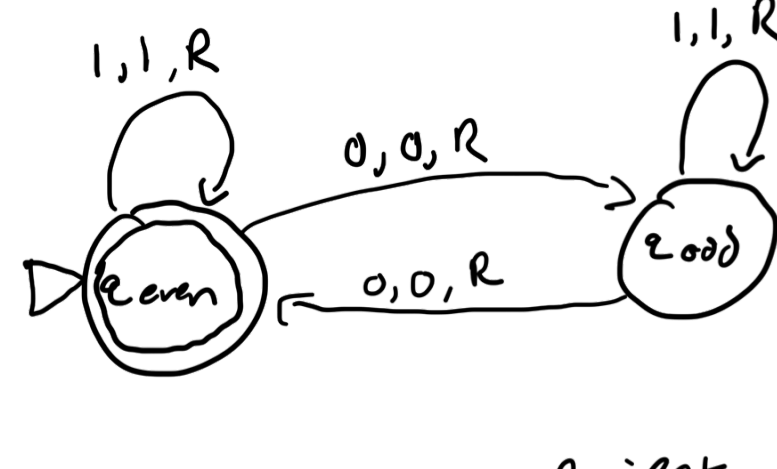
Formal models of computation

- A mathematically precise abstraction of what a computer can do
- Key: model what's computable in principle



Turing machine (TM)

- An infinite tape of cells (each cell holds 0, 1, or blank)
- A tape head that:
 - Reads tape
 - writes symbol
 - moves left or right



110010 $\rightarrow$ Reject
111111 $\rightarrow$ Accept
111010 $\rightarrow$ Accept

Decidability & Halting Problem

- Decision Problem: Yes/no question about an input
 - Given a graph $G=(V,E)$ and vertices (s,t) , is there a path from s to t ?
- Language: set of strings over some alphabet
 - we can view a decision problem as a language: all strings where the answer is yes
- A language (L) is decidable if there exists a TM that:
 - Halts on every input
 - Accepts exactly those strings in L

Halting Problem

- Given a program P and input x , will P ever halt when run on x ?

we can encode "program + input" as a string, so this becomes a language

$$HALT = \{ \langle P, x \rangle \mid \text{Program } P \text{ halts on input } x \}$$

Theorem: The halting problem is undecidable,

Diagonalization proof

1. Assume for contradiction there exists a TM H such that:

- Input: $\langle P, x \rangle$
- output: Yes if P halts on x , no if P runs forever on x
- H itself always halts

2. Use this to build a "weird" machine D

- Input: $\langle P \rangle$
- Run H on $\langle D, P \rangle$
- If H says "yes", D goes into an infinite loop
- If H says "no", D halts

$D(P)$:

if $H(P, P) = \text{"yes"}$:

loop forever

else:

halt

D disagrees with H that predicts about P run on P

Q: what happens when we run $D(D)$?

Case 1: Suppose $D(P)$ halts

- Then $H(D, D)$ must say "yes" $\rightarrow$ by definition of D , this is a contradiction!

Case 2: Suppose $D(D)$ loops forever

- Then $H(D, D)$ must loop forever $\rightarrow$ by definition of D , this is a contradiction!

Time complexity TM

- Given a TM, we define its complexity $T(n)$ as:
 - maximum number of steps it takes to halt on any input of length n
- on real machines we count instructions, on TM, we count transitions, but up to polynomial factors they're equivalent.

Class P

P is the class of decision prob